

Ejercicios iniciales UT2_1

Explicación de cómo he realizado la actividad, qué he aprendido y cómo he comprobado que la página es válida.

Autor: Javier Jimenez Alonso

Curso: 2º DAW

Módulo: DWES · Desarrollo web en entorno servidor

Actividad: Ejercicios iniciales (opcionales) UT2_1 · Lenguaje PHP. Inserción de código en páginas web

Entregable: archivo `DWSEjerciciosIniciales_JimenezAlonsoJavier.zip` con el proyecto (`index.php`) y este documento

1. Qué pedía el enunciado

El enunciado pedía tres ejercicios de PHP y además dos condiciones para todos ellos:

- Usar la estructura básica de HTML que se nos da como ejemplo en el archivo `básico.php`.
- Comprobar que la página valida correctamente en el validador de HTML de W3C.
- Meter los tres ejercicios dentro de pestañas para poder verlos todos en una misma página, en un archivo llamado `index.php`.

Los ejercicios son: (1) generar un número aleatorio y mostrarlo a un tamaño aleatorio entre el 200 % y el 800 %; (2) mostrar un emoticono Unicode aleatorio entre los caracteres 128512 y 128586; y (3) contar las letras «t» minúsculas de la frase «This is a test» y, en un segundo apartado, contar todas las «t».

2. Cómo lo he organizado

2.1. Dónde tiene que estar el archivo para que funcione

PHP no se ejecuta en el navegador, lo ejecuta el servidor web. Por eso el archivo tiene que estar dentro de la carpeta que Apache sirve, que en XAMPP es `C:\xampp\htdocs`. Yo tengo la carpeta del módulo enlazada dentro de `htdocs`, así que la página se abre en `http://localhost/2-DAW/Entorno%20Servidor/UT2/DWSEjerciciosIniciales/index.php` después de pulsar *Start* en Apache dentro del XAMPP Control Panel.

El primer detalle fue el nombre: el archivo me llegó como `básico.php.txt`. Con la extensión `.txt` el navegador lo muestra como texto y el servidor no ejecuta el PHP, así que lo renombré a `básico.php`.

2.2. La estructura HTML y el código incrustado

El archivo `index.php` es el ejemplo `básico.php` con los tres ejercicios dentro del bloque de PHP del cuerpo. Mantengo el `DOCTYPE`, el `lang="es"`, el `meta charset="UTF-8"` (sin él los acentos y la «ñ» se verían mal) y el `meta viewport`, porque son justo los metadatos que pide el enunciado para que la

página valide. El código PHP va delimitado por `<?php` y `?>`: el servidor lee ese bloque, lo ejecuta y devuelve el resultado ya escrito en HTML, que es el código incrustado que se explica en la teoría.

Para escribir en pantalla he usado `echo`, que es la palabra reservada que muestra texto o el valor de una variable, tal y como aparece en el apartado 2 de la teoría. Uso comillas dobles en las cadenas porque así las variables (`$numero`, `$tamano` ...) se sustituyen solas por su valor, y dentro de esas cadenas los atributos HTML van con comillas simples para no romper la cadena de PHP.

2.3. Las pestañas

Para cumplir el apartado de las pestañas he usado el componente *Navs and tabs* de Bootstrap 5, el mismo Bootstrap que ya venía en el ejemplo `básico.php`, enlazado desde un CDN. Copié el HTML del ejemplo de la documentación oficial de Bootstrap (<https://getbootstrap.com/docs/5.3/components/navs-tabs/>) y lo simplifiqué para dejar solo lo imprescindible:

```
<ul class="nav nav-tabs mb-3">
  <li class="nav-item">
    <button class="nav-link active" data-bs-toggle="tab"
      data-bs-target="#panelEjercicio1" type="button">Ejercicio 1</button>
  </li>
</ul>

<div class="tab-content">
  <div class="tab-pane fade show active" id="panelEjercicio1"> ... </div>
</div>
```

- El `<ul class="nav nav-tabs">` es la barra donde se colocan los botones, y cada botón va dentro de un `<li class="nav-item">`.
- `data-bs-toggle="tab"` le dice al JavaScript de Bootstrap que ese botón es una pestaña.
- `data-bs-target="#panelEjercicio1"` indica qué panel hay que mostrar: el que tenga ese `id`. Por eso el identificador del botón y el del panel tienen que escribirse exactamente igual.
- La primera pestaña lleva la clase `active` y su panel `fade show active`, porque es la que se ve al abrir la página.
- El JavaScript de Bootstrap (`bootstrap.bundle.min.js`) va al final del `body` y es el que lee esos dos atributos para cambiar de panel al pulsar.

2.4. Los tres ejercicios

En los tres he usado la función `rand(min, max)`, que es la que explica la teoría en el apartado de funciones numéricas (`rand(1, 9)`). Cada vez que recargo la página el servidor vuelve a ejecutar el archivo y por eso los valores cambian.

Ejercicio 1. Genero el número y el tamaño por separado y el tamaño lo meto en un atributo `style` del párrafo:

```
$numero = rand(1, 1000);
$tamano = rand(200, 800);

echo "<h2 class='h4 text-primary'>Ejercicio 1: número aleatorio</h2>";
```

```
echo "<p style='font-size: $tamano%;'>$numero</p>";
echo "<hr>";
echo "<p class='text-secondary small'>Tamaño de la letra: $tamano%</p>";
```

Ejercicio 2. El emoticono no se puede escribir directamente en el teclado porque es un carácter Unicode, así que se muestra con su código numérico con el formato `😀`. Como el código lo genera `rand()`, escribo el signo `&#` delante de la variable y el punto y coma detrás:

```
$emoticono = rand(128512, 128586);

echo "<p style='font-size: 100px;'>&#$emoticono;</p>";
echo "<hr>";
echo "<p class='text-secondary small'>Código Unicode: $emoticono</p>";
```

Ejercicio 3. Uso `substr_count($texto, $subcadena)`, que cuenta cuántas veces aparece una subcadena dentro de un texto. Como esa función distingue mayúsculas de minúsculas, para el apartado 2 sumo también la cuenta de la «T» mayúscula:

```
$frase = "This is a test";
$minusculas = substr_count($frase, "t");
$todas = $minusculas + substr_count($frase, "T");
```

El resultado es **2** t minúsculas (las dos de «test») y **3** si contamos todas, porque al principio de «This» hay una «T» mayúscula. Esa es la respuesta que escribe el enunciado en el apartado 2, y la muestro también en texto en la página.

3. Comprobación del funcionamiento

Estas capturas son del navegador pulsando realmente los botones de las pestañas, no son imágenes del código.

Pestaña 1: número aleatorio (la que aparece al abrir la página)

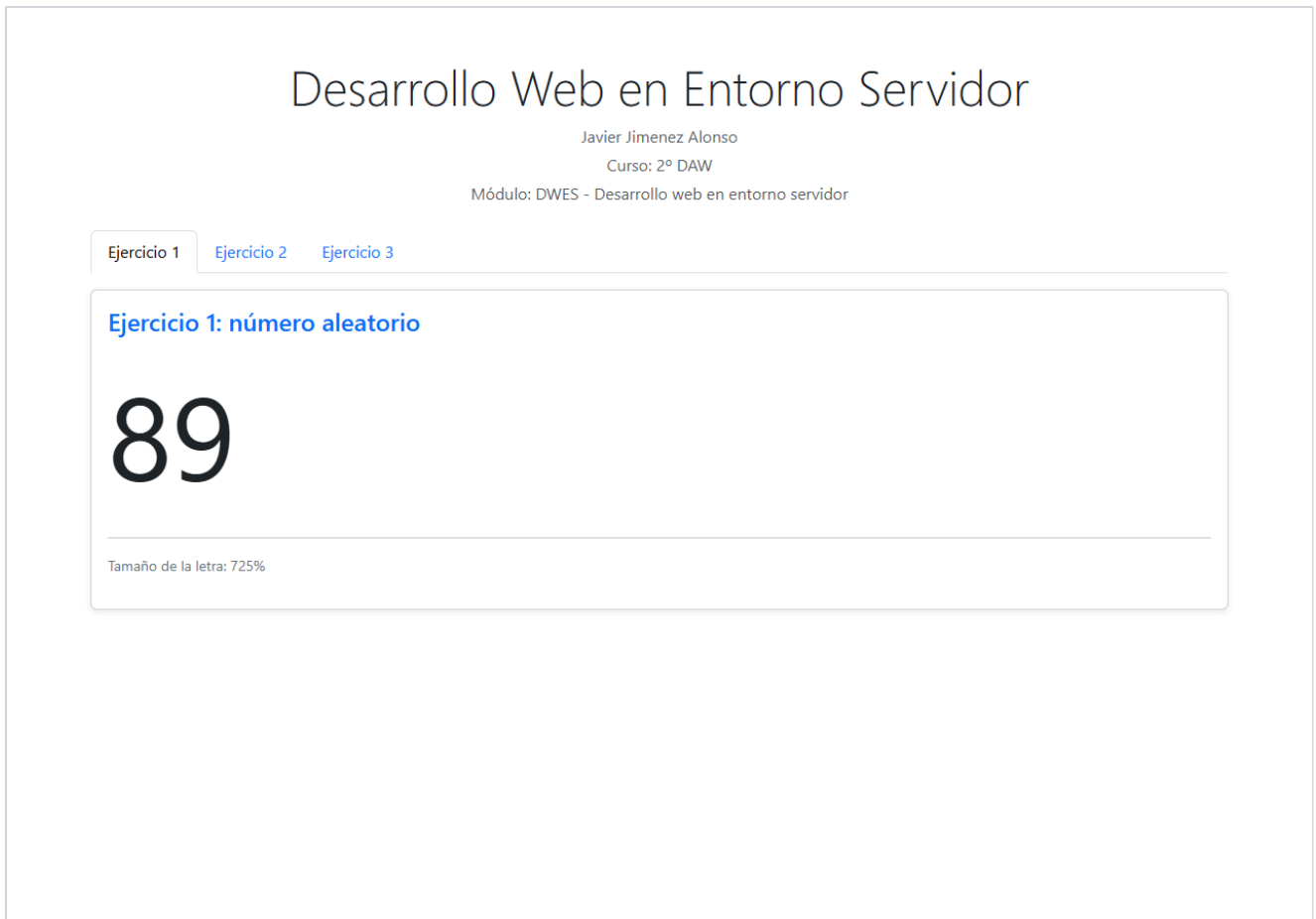


Figura 1. Ejercicio 1 con un número y un tamaño generados al azar (en esta ejecución, 765 %).

Pestaña 2: emoticono Unicode

Desarrollo Web en Entorno Servidor

Javier Jimenez Alonso

Curso: 2º DAW

Módulo: DWES - Desarrollo web en entorno servidor

[Ejercicio 1](#)

Ejercicio 2

[Ejercicio 3](#)

Ejercicio 2: emoticono Unicode



Código Unicode: 128561

Figura 2. Ejercicio 2. El emoticono sale del rango Unicode 128512–128586, que es el que pide el enunciado.

Pestaña 3: conteo de la letra «t»

Desarrollo Web en Entorno Servidor

Javier Jimenez Alonso

Curso: 2º DAW

Módulo: DWES - Desarrollo web en entorno servidor

[Ejercicio 1](#) [Ejercicio 2](#) [Ejercicio 3](#)

Ejercicio 3: contar la letra 't'

Frase analizada: *This is a test*

1) Número de 't' minúsculas: **2**

2) Número de todas las 't': **3**

Para contar todas hay que sumar también las mayúsculas.

Figura 3. Ejercicio 3: 2 t minúsculas y 3 t contando también las mayúsculas.

4. Validación en el validador de W3C

Para validar la página he usado el Nu Html Checker, la herramienta oficial de W3C (<https://validator.w3.org/nu/>). Como el validador no ejecuta PHP, lo que se sube es el HTML que genera el servidor, que es el que se ve al pulsar *Ctrl + U* (ver código fuente) en el navegador. Los pasos son:

1. Abro la página en el navegador y copio el código fuente que devuelve el servidor.
2. En el validador elijo la opción *file upload* y subo ese archivo HTML.
3. Pulso *Check* y leo el resultado.

El resultado es que el documento no tiene ni errores ni avisos, es decir, la página valida correctamente tal y como pide el enunciado.

Nu Html Checker

This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change

Showing results for `html_generado.html` (checked with vnu 26.9.16)

Checker Input

Show ☐ source ☐ outline ☐ image report ☐ errors & warnings only Options...

Check by file upload Elegir archivo No se ha seleccionado ningún archivo

Uploaded files with `.xhtml` or `.xht` extensions are parsed using the XML parser.

Check

Document checking completed. No errors or warnings to show.

Used the HTML parser.
Total execution time 5 milliseconds.

[About this checker](#) • [Report an issue](#)

Figura 4. Resultado de la validación: «Document checking completed. No errors or warnings to show».

Comprobé también que los atributos `data-bs-toggle` y `data-bs-target`, y los comentarios que hay en el código, no provocan ningún error de validación, porque los atributos que empiezan por `data-` están permitidos en HTML5.

5. Problemas que me he encontrado

- **Las pestañas no cambiaban de panel.** Al principio pulsaba los botones y seguía viéndose el ejercicio 1. Mirando la consola del navegador vi el error *Illegal invocation* de Bootstrap. El motivo era que había borrado la etiqueta de apertura `<ul class="nav nav-tabs mb-3">` y me había quedado solo el `</ul>`, con lo cual los botones se quedaban sueltos dentro del `<main>` y Bootstrap no encontraba la lista de pestañas. Al volver a poner el `<ul>` las pestañas ya funcionaron. Es un ejemplo de que conviene comprobar la consola del navegador cuando algo no responde.
- **El archivo con extensión `.txt`.** No se ejecutaba porque el servidor no lo reconocía como PHP.
- **El emoticono.** Al principio iba a escribir el símbolo directamente y no se podía, por eso uso su código Unicode numérico.
- **Los acentos.** Con el `meta charset="UTF-8"` bien puesto, «básico» y los textos en español se ven correctamente.

6. Lo que he aprendido

- La diferencia entre ejecutar el código en el cliente y en el servidor: el navegador no entiende PHP, lo entiende el servidor web (Apache), por eso el archivo tiene que estar en `htdocs` y abrirse con `http://localhost`.
- El código incrustado: escribir HTML y meter dentro bloques `<?php ... ?>` que el servidor sustituye por su salida.
- A usar variables, comillas dobles con interpolación y la instrucción `echo`.
- Las funciones nativas `rand()` y `substr_count()`, y que `substr_count()` distingue mayúsculas de minúsculas.
- La importancia de la cabecera de un documento HTML (`DOCTYPE`, `lang`, `meta charset`) y de validarlo con la herramienta de W3C.
- Que los atributos `data-*` de HTML5 son la forma de que una biblioteca como Bootstrap añada comportamiento sin Javascript propio, y que para eso hace falta cargar su `bundle`.
- A usar la consola del navegador y el inspector para depurar, que es donde vi el error de Bootstrap.

7. Conclusión

Con esta actividad he visto cómo un archivo HTML puede incluir código PHP que se ejecuta en el servidor, cómo se muestran datos generados al azar y cómo se depura con las herramientas del navegador. Los tres ejercicios están resueltos en un único `index.php`, dentro de pestañas, y la página valida sin errores ni avisos en el validador de W3C, que es lo que pedía el enunciado.